AMS Round 2 Syllabus

Ascent 2026 · Team Round

Contents

Level 1 Language basics **2**

Level 2 Objects and lifetime **2**

Level 3 Standard library **3**

Level 4 Templates and compile time **4**

Level 5 Performance and hardware **4**

Level 6 Advanced **5**

Level 7 External libraries **6**

Out of scope **6**

Level 1 Language basics

1 Types and declarations

fundamental types, `<cstdint>` fixed-width integers, integer promotion, integer conversion, narrowing, signed and unsigned, signed overflow, `auto`, `decltype`, aggregate initialisation, designated initialisers, list initialisation, the most vexing parse, `enum`, `enum class`, `using` aliases, `typedef`

2 const and cv-qualifiers

`const`, `mutable`, const member functions, const overloads, pointer-to-const, const pointer, top-level const, low-level const, `volatile`

3 Functions and name lookup

overload resolution, default arguments, argument-dependent lookup, namespaces, `using` declarations, `using` directives, name hiding, `explicit`, converting constructors, deleted functions, trailing return types

4 Classes and special members

constructors, destructors, member initialiser lists, initialisation order, rule of zero, rule of three, rule of five, copy constructor, copy assignment, defaulted members, deleted members, `static` members, nested types, friends

5 Inheritance and polymorphism

virtual functions, virtual destructors, `override`, `final`, abstract classes, pure virtual functions, object slicing, vtables, multiple inheritance, the diamond problem, virtual inheritance, inheritance access specifiers

6 Casts and RTTI

`static_cast`, `dynamic_cast`, `const_cast`, `reinterpret_cast`, `std::bit_cast`, C-style casts, `typeid`, `std::type_info`

Level 2 Objects and lifetime

7 Lifetime and storage

automatic storage, static storage, thread storage, dynamic storage, the stack, the heap, static initialisation order, `constinit`, temporaries, lifetime extension by const reference, dangling

8 Move semantics

value categories, lvalue, rvalue, xvalue, glvalue, prvalue, move constructor, move assignment, `std::move`, `std::forward`, forwarding references, perfect forwarding, reference collapsing, copy elision, RVO, NRVO, moved-from state

9 Ownership

`std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`, `std::make_unique`, `std::make_shared`, custom deleters, control block layout, reference cycles, raw pointers as non-owning handles, `enable_shared_from_this` *

10 RAII and exception safety

RAII, scope guards, no-throw guarantee, strong guarantee, basic guarantee, copy-and-swap, ADL `swap`, `noexcept`, `noexcept` on move operations, stack unwinding, throwing destructors, exception cost

11 Error handling

`std::optional`, `std::expected`, `and_then`, `or_else`, `transform`, `value_or`, error enums, error precedence, `std::error_code`, exceptions against return values, total functions, partial functions, `std::unreachable`

12 Operators and comparison

operator overloading, member and free operators, `operator<=>`, `std::strong_ordering`, `std::weak_ordering`, `std::partial_ordering`, `std::is_lt` and family, defaulted comparisons, `std::cmp_less` and family, `std::in_range`, strict weak ordering

13 Utility types

`std::pair`, `std::tuple`, `std::tie`, `std::apply`, structured bindings, `std::variant`, `std::visit`, `std::any`, `std::reference_wrapper`, `std::to_underlying`, `std::byte`

14 Lambdas and callables

closures, capture by value, capture by reference, init-captures, `mutable` lambdas, generic lambdas, template lambdas, `std::function`, `std::move_only_function`, `std::invoke`, `std::bind_front`, `std::bind_back`, projections

15 Separate compilation

headers, source files, include guards, `#pragma once`, forward declarations, include weight, the preprocessor, macros, header compile cost

Level 3 Standard library

16 Containers

`std::vector`, `std::array`, `std::deque`, `std::list`, `std::forward_list`, `std::map`, `std::set`, `std::multimap`, `std::multiset`, `std::unordered_map`, `std::unordered_set`, complexity guarantees, iterator invalidation rules, `insert`, `emplace`, `try_emplace`, node handles

17 Container internals

vector growth, `reserve`, `capacity`, `size`, `shrink_to_fit`, `std::string` small-string optimisation, red-black trees, buckets, load factor, rehashing, `std::hash`, custom hash functions, deque block structure

18 Iterators

iterator categories, input, forward, bidirectional, random-access, contiguous, `std::iterator_traits`, `begin`, `end`, sentinels, `std::advance`, `std::distance`, insert iterators, stream iterators, `std::reverse_iterator`, writing an iterator

19 Algorithms

`sort`, `stable_sort`, `partial_sort`, `nth_element`, `partition`, `stable_partition`, `rotate`, `unique`, `reverse`, `shuffle`, `min_element`, `max_element`, `minmax_element`, `merge`, `inplace_merge`, `set_union`, `set_intersection`, `set_difference`, `all_of`, `any_of`, `none_of`, `count_if`, `transform`, `accumulate`, `for_each`, `adjacent_find`, `remove_if`, the erase-remove idiom

20 Searching and ordering

`std::lower_bound`, `std::upper_bound`, `std::equal_range`, `std::binary_search`, hand-written binary search, custom comparators, comparator consistency, comparison sorts, counting sort, radix sort, stability

21 Strings and views

`std::string`, `std::string_view`, `std::span`, `substr`, `starts_with`, `ends_with`, `contains`, dangling views, views into locals, character encoding, `char8_t *`, locales, `std::regex`

22 Parsing and formatting

`<charconv>`, `std::from_chars`, `std::to_chars`, `std::format`, `std::format_to`, `std::formatter` specialisation, formatting into a reserved buffer, `std::stod`, `std::stoi`, `std::ostringstream`, `std::endl`, `'\n'`, `sync_with_stdio`, buffered I/O, unbuffered I/O

23 Numerics library

`accumulate`, `reduce`, `transform_reduce`, `inclusive_scan`, `exclusive_scan`, `partial_sum`, `gcd`, `lcm`, `midpoint`, `<cmath>`, `std::fma`, `std::hypot`, `<random>` engines, distributions, seeding, `<chrono>` clocks, durations, `steady_clock`, calendar types

24 Ranges and views

`views::filter`, `views::transform`, `views::take`, `views::take_while`, `views::drop`, `views::iota`, `views::reverse`, `views::join`, `views::split`, `views::zip`, `views::enumerate`, `views::chunk`, `views::chunk_by`, `views::slide`, `views::pairwise`, `views::stride`, `views::adjacent_transform`, `views::cartesian_product`, `ranges::to`, `ranges::fold_left`, range algorithms, laziness, borrowed ranges, dangling ranges

Level 4 Templates and compile time

25 Templates

function templates, class templates, member templates, type parameters, non-type template parameters, default template arguments, variadic packs, pack expansion, fold expressions, `sizeof...`, explicit specialisation, partial specialisation, class template argument deduction, two-phase lookup, dependent names, `typename` and `template` disambiguation

26 Deduction and traits

template argument deduction, `auto`, `decltype(auto)`, `decltype`, `std::declval`, `<type_traits>`, `is_same`, `is_base_of`, `conditional`, `remove_cvref`, `decay`, `void_t`, SFINAE, `std::enable_if`, tag dispatch, detection idioms

27 Concepts

`concept`, `requires` clauses, requires-expressions, `integral`, `floating_point`, `invocable`, `convertible_to`, `same_as`, `derived_from`, `ranges::range` and family, subsumption, partial ordering by constraints, constraints against SFINAE

28 Compile-time computation

`constexpr` functions, `constexpr` variables, `consteval`, `if constexpr`, `static_assert`, constant evaluation rules, `constexpr` containers, compile-time lookup tables, compile-time strings *, CRTP *, policy-based design *, type lists *

Level 5 Performance and hardware

29 Data layout

struct layout, alignment, padding, `alignof`, `alignas`, `offsetof`, `sizeof`, hot struct packing, array of structs, struct of arrays, empty base optimisation, `[[no_unique_address]]` *

30 Memory hierarchy

cache lines, L1, L2, L3, spatial locality, temporal locality, pointer chasing, contiguous traversal, memory bandwidth, prefetching, TLB *, false sharing *

31 Allocation

cost of `new`, `reserve`, growth policies, amortised analysis, buffer reuse across calls, `clear()` without release, arena allocation, pool allocation, `std::pmr`, `monotonic_buffer_resource`, custom allocators *, allocator propagation *

32 Dispatch and indirection

branch prediction, branch misprediction, branchless selection, sorting for predictability, `[[likely]]`, `[[unlikely]]`, virtual dispatch cost, devirtualisation, function pointers, `std::function` against template parameters, type erasure cost, inlining, inlining barriers

33 Floating point

IEEE 754, `std::numeric_limits`, machine epsilon, non-associativity of addition, catastrophic cancellation, loss of significance, expression reformulation, relative tolerance, absolute tolerance, NaN, infinity, signed zero, `std::nextafter`, `std::isfinite`, division cost, multiplication by reciprocal, `std::pow` against `x*x`, compensated summation *

34 Bit manipulation

`std::popcount`, `std::countl_zero`, `std::countr_zero`, `std::bit_width`, `std::has_single_bit`, `std::bit_ceil`, `std::bit_floor`, `std::rotl`, `std::rotr`, `std::byteswap`, `std::endian`, masks, shifts, bit packing, shift-and-mask against division

35 The optimiser

`-O0`, `-O1`, `-O2`, `-O3`, `-Os`, inlining, common subexpression elimination, dead code elimination, loop-invariant code motion, strength reduction, loop unrolling, auto-vectorisation, vectorisation barriers, pointer aliasing, floating-point reordering, `-ffast-math`, reading generated assembly *

36 Benchmarking

wall time, CPU time, `steady_clock`, `system_clock`, median, mean, variance, run-to-run noise, warm cache, cold cache, dead code elimination in benchmarks, retired instructions, flame graphs, self time, total time, Amdahl's law

Level 6 Advanced

37 Linkage and the ODR

declaration, definition, the One Definition Rule, ODR violations, internal linkage, external linkage, anonymous namespaces, file-scope `static`, `inline` functions, `inline` variables, `extern`, `extern "C"`, vague linkage, symbol collisions, name mangling, linker errors against compiler errors, ABI stability

38 Interface design

interface and implementation separation, the pimpl idiom, `unique_ptr<Impl>`, shared immutable state with `shared_ptr<const T>`, constructor-time work against per-call work, value semantics across a boundary, reference semantics across a boundary, preconditions, postconditions, invariants, substitutable implementations

39 Writing data structures

flat sorted vector against tree, open addressing, linear probing, chaining, tombstones, intrusive containers, ring buffers, free lists, small-buffer optimisation, `make_heap`, `push_heap`, `pop_heap`, `std::priority_queue`, updatable heaps, indexed heaps, allocator-aware containers *

40 Undefined behaviour

undefined behaviour, unspecified behaviour, implementation-defined behaviour, erroneous behaviour, signed overflow, out-of-bounds access, strict aliasing, type punning, alignment violations, use-after-free, use-after-move, dangling references, uninitialised reads, sequencing, evaluation order, `std::launder` *

41 Determinism

bit-identical output, order of operations, container iteration order, hash ordering, platform differences, compiler differences, reference answers within tolerance, independent implementations of one specification

Level 7 External libraries

42 nlohmann/json

`json::parse`, `at`, `operator[]`, `contains`, `is_object`, `is_array`, `is_number`, `is_string`, `get<T>`, `dump`, `ordered_json`, the SAX interface, `json::sax_parse`, custom SAX handlers, node allocation cost

43 Boost.Unordered

`unordered_flat_map`, `unordered_flat_set`, `unordered_node_map`, `unordered_node_set`, open addressing, `max_load_factor`, `reserve`, custom hash and equality, heterogeneous lookup

44 Boost.Container

`small_vector`, `static_vector`, `flat_map`, `flat_set`, `flat_multimap`, `stable_vector`, `devector *`, reserve semantics, ordered-range construction

45 Boost.Math

`boost::math::constants`, special functions, `interpolators::pchip`, `interpolators::cardinal_cubic_b_spline`, `interpolators::barycentric_rational`, `interpolators::makima *`, `tools::roots` (`bisect`, `newton_raphson_iterate`, `toms748_solve`), `quadrature::trapezoidal`, `quadrature::gauss`, `quadrature::tanh_sinh`

46 Boost.Multiprecision

`cpp_int`, `int128_t`, `uint128_t`, `cpp_rational`, `cpp_bin_float`, `cpp_dec_float`, fixed-width against arbitrary-width backends, `numerator`, `denominator`, conversion to and from built-in types, `number<>` expression templates

47 Boost.Graph

`adjacency_list`, `adjacency_matrix`, `compressed_sparse_row_graph`, vertex descriptors, edge descriptors, `add_vertex`, `add_edge`, `vertices`, `edges`, `out_edges`, property maps, `get`, `put`, bundled properties, `breadth_first_search`, `depth_first_search`, visitors, `topological_sort`, `dijkstra_shortest_paths`, `bellman_ford_shortest_paths`, `strong_components`, `connected_components`, `kruskal_minimum_spanning_tree`

Out of scope

Concurrency	threads, `std::atomic`, mutexes, `std::async`, coroutines
Language	modules
Tooling	CMake, build systems, `gdb`, `perf`
Low level	intrinsics, inline assembly, `-ffast-math`
Domains	networking, GUI, databases